Laboratory 3
Variant 1
Group A
By Meloni and Abahumna

1 Introduction

The task was to minimize the Group A objective function with a genetic algorithm:

$$f(x_1, x_2) = 1.5 - e^{-x_1^2 - x_2^2} - 0.5e^{-(x_1-1)^2-(x_2+2)^2}. \tag{1}$$

The function is smooth and multimodal. The deepest minimum is close to the origin, while another broader basin is created near $(1, -2)$. This makes the problem suitable for a genetic algorithm because the search method must balance exploration and exploitation instead of following only one local descent direction.

Genetic algorithms are usually used when the search space is irregular, non-convex, discontinuous, or when gradient information is unavailable. Their main advantages are robustness and the ability to explore multiple regions at the same time. Their main disadvantages are stochastic variability, dependence on hyperparameters, and a higher number of function evaluations than a deterministic method on a simple smooth problem.

2 Implementation

The submitted program keeps the template structure with the `Chromosome` and `GeneticAlgorithm` classes. Each chromosome stores a binary string of length 32, so each of the two decision variables is encoded with 16 bits. Decoding uses min-max normalization:

$$x_i = \ell_i + \frac{d_i}{2^{16} - 1}(u_i - \ell_i), \tag{2}$$

where d_i is the decoded integer and $[\ell_i, u_i]$ is the area of interest. The range $[-4, 4]$ was used for both variables.

The algorithm works in the following order:

1. generate the initial population,
2. decode all chromosomes and evaluate the objective function,
3. store the best individual of the current generation,
4. select parents with tournament selection,

5. apply one-point crossover with probability p_c,
6. mutate one random bit with probability p_m,
7. repeat for the required number of generations.

The parameter study was run with 8 seeds for every tested configuration. For each parameter value, the code stores the best result from every run, computes the mean and sample standard deviation, and picks one representative run whose result is closest to the mean. The visualization for each parameter is saved as one combined figure: the top-left panel contains the summary plot and the remaining three panels contain contour traces for the tested values.

Listing 1 shows the decoding step. Listing 2 shows how cached fitness values are reused in tournament selection.

Listing 1: Chromosome decoding

```
def decode(self, lower_bound, upper_bound, aoi):
    segment = self.array[lower_bound:upper_bound]
    decimal_value = 0
    for bit in segment:
        decimal_value = (decimal_value << 1) | int(bit)

    return min_max_norm(decimal_value, 0, (2 ** len(segment)) - 1,
       aoi[0], aoi[1])
```

Listing 2: Tournament selection with cached fitness

```
def tournament_selection(self, fitness_values):
    parents = []
    for _ in range(self.population_size):
        tournament_indices = random.sample(range(self.
           population_size), self.tournament_size)
        winner_idx = min(tournament_indices, key=lambda idx:
           fitness_values[idx])
        parents.append(Chromosome(self.chromosome_length, self.
           population[winner_idx].array))
    return parents
```

Table 1: Effect of mutation probability on the best objective value over 8 runs.

Mutation probability	Mean best value	Standard deviation
0.00	0.576697826	0.172535849
0.05	0.496604093	0.000025263
0.15	0.496601397	0.000027405

Figure 1 presents the complete mutation study. The summary panel shows the averages and standard deviations from Table 1, while the contour panels show representative traces for the three tested mutation probabilities.

Figure 2 shows the same structure for tournament size. The figure makes it easy to compare both the numerical stability and the search trajectory for each tested value.

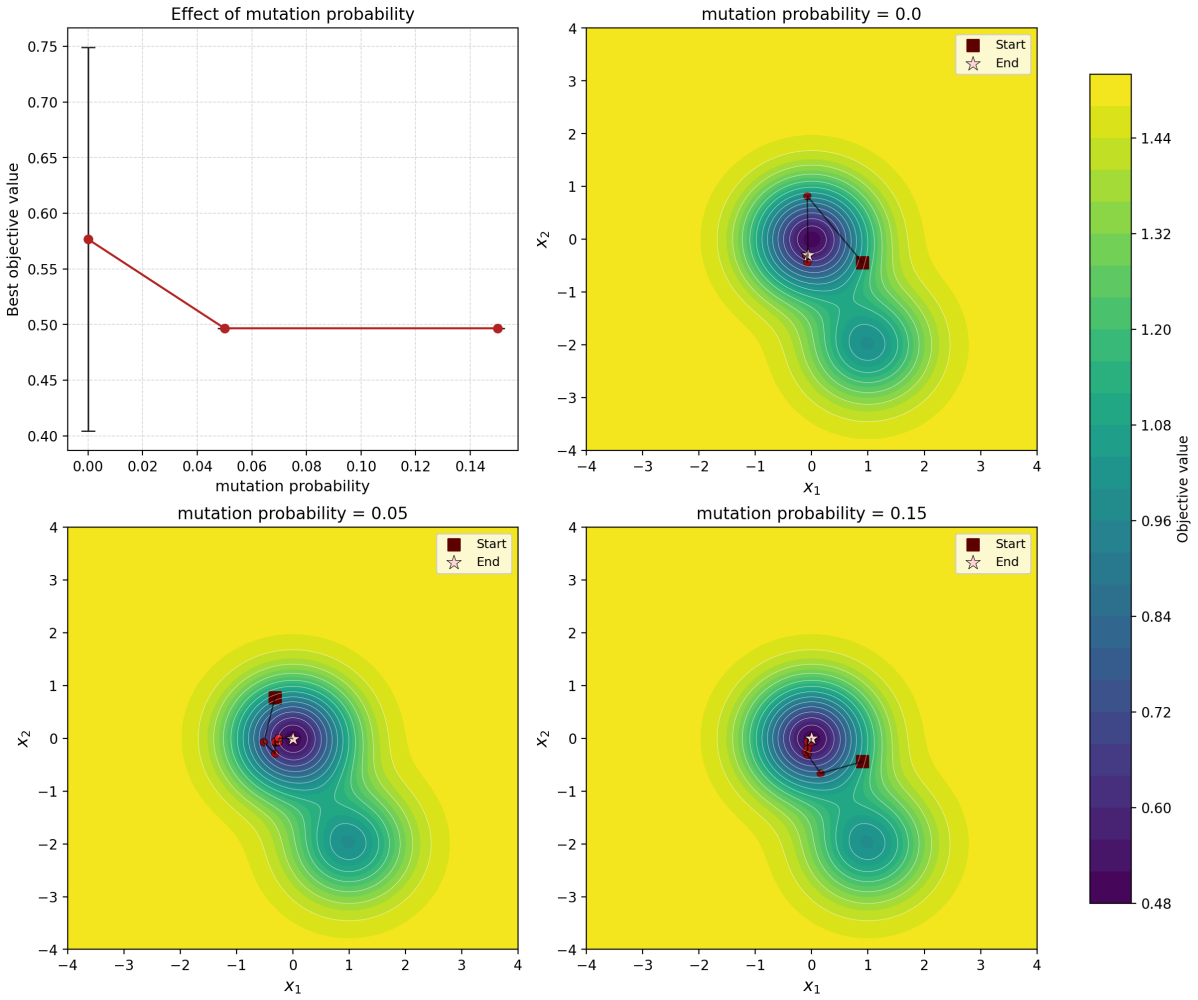


Figure 1: Combined mutation-probability study with one summary panel and three representative contour traces.

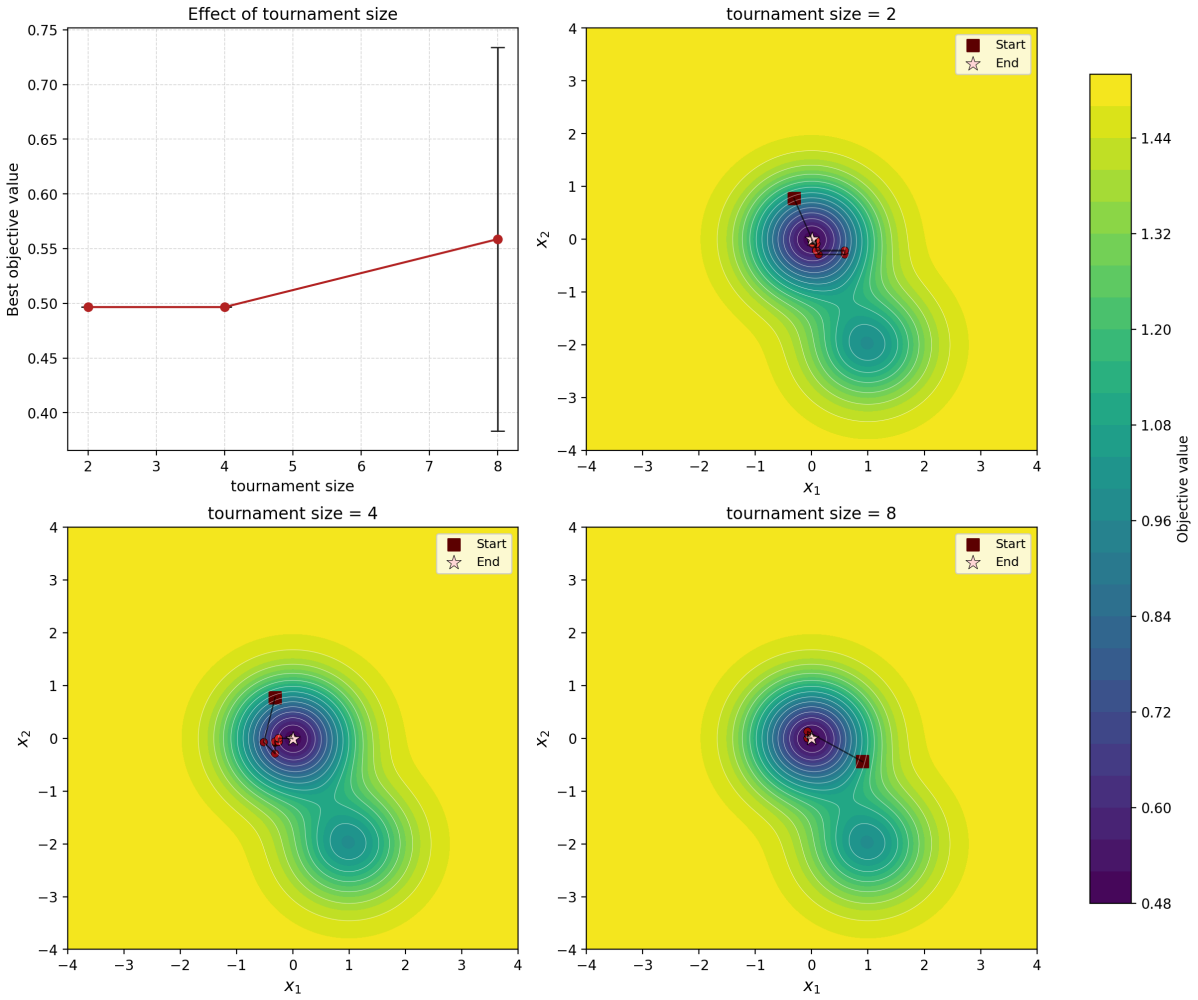


Figure 2: Combined tournament-size study with one summary panel and three representative contour traces.

Table 2: Effect of tournament size on the best objective value over 8 runs.

Tournament size	Mean best value	Standard deviation
2	0.496603130	0.000027045
4	0.496604093	0.000025263
8	0.558643096	0.175402504

Table 3: Effect of population size on the best objective value over 8 runs.

Population size	Mean best value	Standard deviation
20	0.665393619	0.247076974
60	0.496604093	0.000025263
120	0.496602526	0.000029180

Figure 3 shows the population-size study. The weak performance at population size 20 is visible both in the table and in the contour trace, where the best individual remains far from the best region more often.

Figure 4 shows the crossover study. The summary panel confirms that crossover is useful on this problem, while the contour traces show that good crossover settings move the population more reliably toward the narrow minimum near the origin.

3 Discussion

An independent dense grid search on $[-4, 4]^2$ gives a reference minimum close to $(0.003476, -0.006952)$ with objective value 0.49657248. The best run found by the genetic algorithm reached 0.49657249 with tournament size 2, which is practically identical to the reference value at the reported precision.

The baseline configuration ($p_m = 0.05, k = 4, N = 60, p_c = 0.8$) produced a mean best value of 0.49660409 with standard deviation 2.53×10^{-5}. This means the average gap to the reference minimum was about 3.16×10^{-5}, so the algorithm was both accurate and stable under the chosen baseline settings.

Mutation had the clearest threshold effect. With no mutation, the mean best value increased to 0.57669783 and the standard deviation rose to 0.17253585. Some runs stayed near the secondary basin around $(1, -2)$ instead of escaping toward the global minimum region. Once mutation was enabled, the behavior improved sharply. Mutation probabilities 0.05 and 0.15 both gave very strong and very stable results, so moderate mutation was enough for this landscape.

Tournament size controlled selection pressure. Sizes 2 and 4 were both stable and

Table 4: Effect of crossover probability on the best objective value over 8 runs.

Crossover probability	Mean best value	Standard deviation
0.00	0.558657672	0.175416468
0.40	0.496649695	0.000172603
0.80	0.496604093	0.000025263

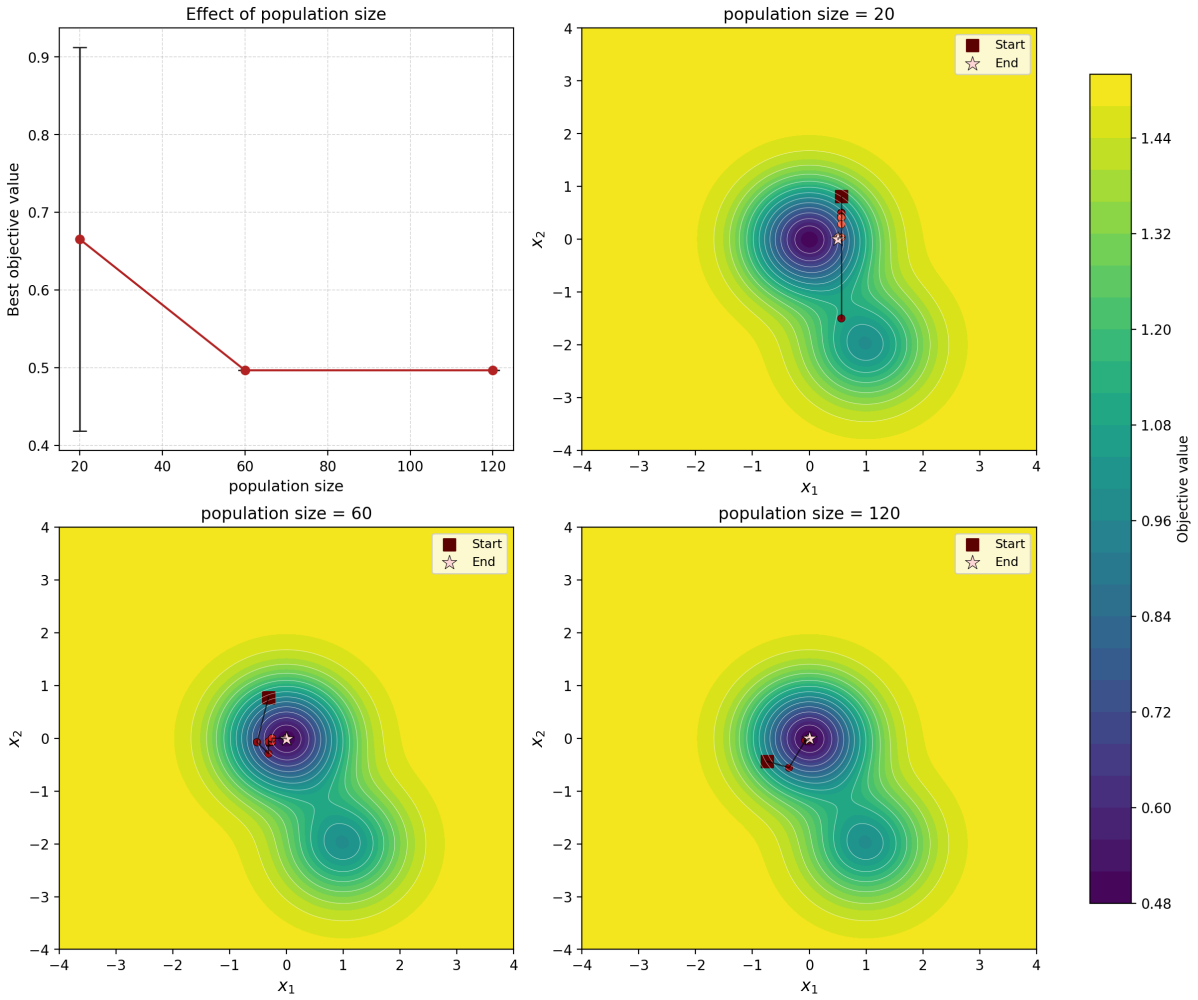


Figure 3: Combined population-size study with one summary panel and three representative contour traces.

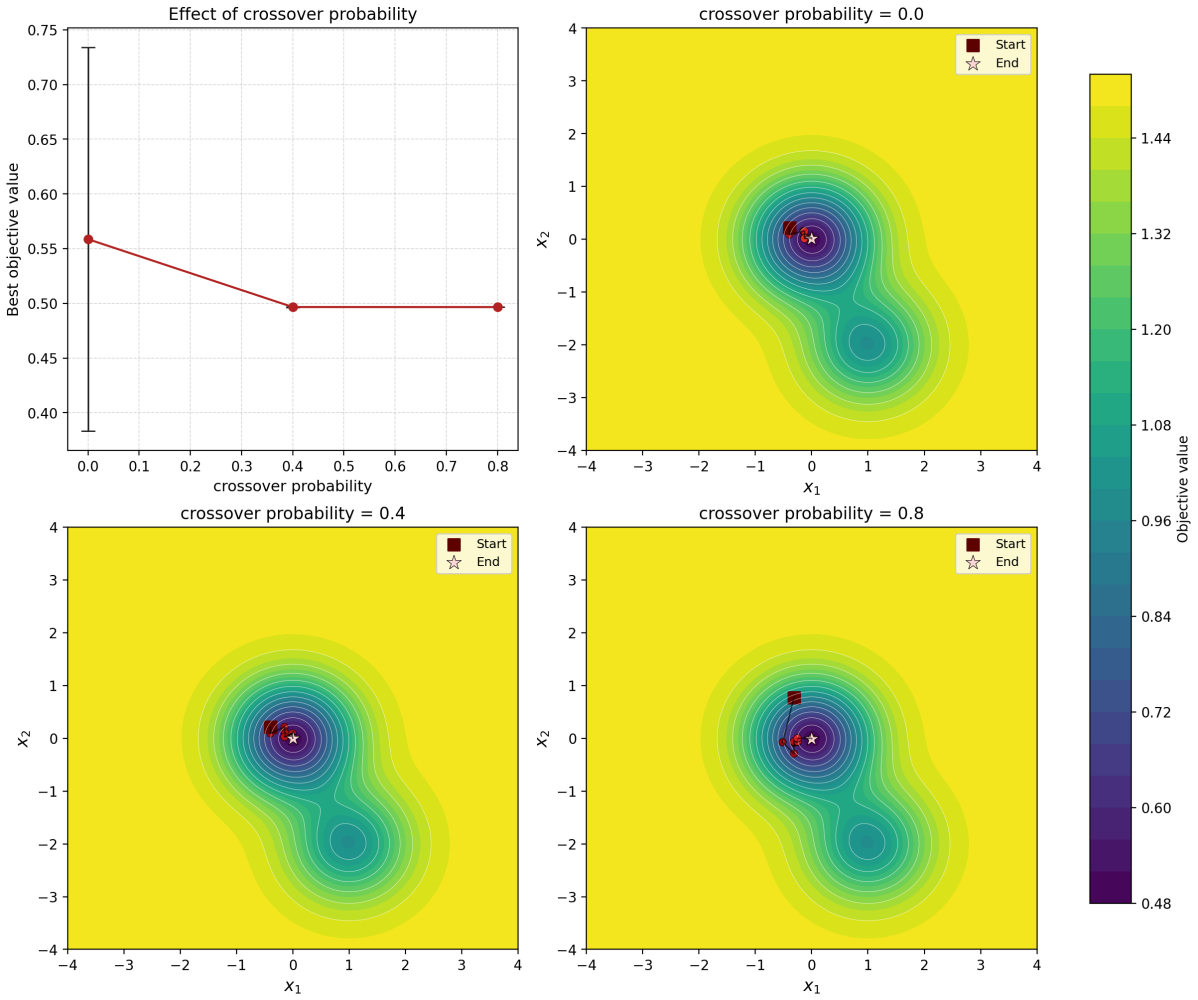


Figure 4: Combined crossover-probability study with one summary panel and three representative contour traces.

accurate. Tournament size 8 performed much worse, with mean 0.55864310 and standard deviation 0.17540250. This is consistent with premature convergence: the selection pressure becomes so strong that diversity disappears too early and the population can no longer recover from a bad basin.

Population size had the strongest effect on reliability. With only 20 chromosomes, the mean best value was 0.66539362 and the standard deviation was 0.24707697. With 60 and 120 chromosomes, the results became very stable and stayed close to the reference minimum. Increasing the population from 60 to 120 gave only a very small improvement, so population size 60 was already sufficient.

Crossover was also important. Without crossover, the mean best value was 0.55865767. At $p_c = 0.4$ the result improved strongly, and at $p_c = 0.8$ the mean dropped to 0.49660409 with a very small standard deviation. On this problem, crossover clearly helped combine useful partial solutions and move the best individual toward the correct basin more consistently.

The study answers the task requirements directly. Mutation is necessary, crossover is beneficial, tournament pressure should remain moderate, and the population must be large enough to preserve diversity. A practical configuration for this function is population size 60, tournament size 2 or 4, mutation probability around 0.05, and crossover probability around 0.8.

4 Conclusion

The experiment showed that a simple binary genetic algorithm can solve the Group A optimization problem very accurately when its operators are balanced correctly. The most important lesson was that reliability depends less on one single operator and more on the balance between diversity and selection pressure.

The most difficult part was the parameter study, because weak settings can still look acceptable in a single run. Repeating each configuration multiple times made the differences much clearer. If the project were extended further, useful improvements would be elitism, adaptive mutation, or a real-valued representation, but they were not necessary for the required lab objectives.